Build a Production-Ready React AI Framework

Objective

Create an open-source AI framework similar to Superpowers, but specialized for building production-ready React applications from nothing.

The framework must work with any AI coding agent, including:

- Claude Code
- Codex
- Cursor
- Windsurf
- Antigravity
- Future AI coding tools

The framework must guide developers through a structured software development lifecycle rather than simply generating code.

The framework should behave like a Technical Architect + Engineering Lead + Project Setup Assistant.

The primary goal is:

"Help developers convert requirement documents into a production-ready React application using a guided, step-by-step workflow."

Core Philosophy

The framework must enforce a strict workflow.

Developers should not be allowed to start feature development before project foundation setup is completed.

Required workflow:

Analyze Requirements
↓
Setup Project Foundation
↓
Build Features
↓
Update Features
↓

Integrate External Services
↓
Review Project
↓
Generate Tests
↓
Production Ready Application

The framework must be opinionated and encourage enterprise-grade architecture.

Command System

The framework must expose the following commands.

/analyze-project

Purpose:

Analyze project requirements and generate implementation plans.

Input:

- SRS
- BRD
- MOM
- Product Requirement Documents
- Figma Notes
- Existing Documentation
- Existing Project Codebase

Behavior:

1. Search for all available documentation.

2. Analyze documentation.

3. Extract:

4. Functional Requirements

5. Non-Functional Requirements
6. User Roles
7. Features

8. API Requirements
9. Security Requirements
10. Architecture Requirements
11. Automatically create:

project-setup/
feature-plans/

---

project-setup Folder

Generate:

project-setup/
architecture.md
folder-structure.md
routing.md
authentication.md
state-management.md
api-strategy.md
error-handling.md
testing-strategy.md
coding-standards.md
deployment.md

Each file must contain detailed implementation instructions.

---

feature-plans Folder

Generate one file per feature:

feature-plans/
patient-dashboard.md
appointment-management.md
billing.md
notifications.md

Each feature file must contain:

- Business Goal
- User Stories
- Acceptance Criteria
- UI Requirements
- API Requirements

- Validation Rules
- State Management Requirements
- Error Handling Requirements
- Testing Requirements
- Technical Notes

Missing Documentation Handling

If no documentation exists:

Ask:

Do you have:

- SRS?
- MOM?
- BRD?
- Product Notes?
- Existing Screens?

If user selects NO:

Generate:

project-setup/

with default enterprise React architecture.

Generate:

feature-plans/

as empty.

Then recommend:

/setup-project-foundation

After analysis completes, display:

Analysis Complete

Created:
✓ project-setup
✓ feature-plans

Next Required Step:

/setup-project-foundation

Feature development is currently locked.

/setup-project-foundation

Purpose:

Create the complete production-ready React project foundation.

This command is mandatory.

Feature development cannot begin until this command succeeds.

Responsibilities

Create:

- React
- TypeScript
- Vite

Configure:

- ESLint
- Prettier
- Husky
- Lint Staged

Setup:

- Routing
- API Layer
- Authentication Architecture
- State Management
- Error Handling
- Environment Variables
- Logging

- Testing Infrastructure

Generate:

FOUNDATION_COMPLETE.md

Folder Structure Example

src/

app/
routes/
pages/
components/
features/
services/
hooks/
layouts/
store/
constants/
types/
utils/
assets/
styles/
tests/

The exact structure should come from project-setup documentation.

After completion:

Display:

Project Foundation Complete

Architecture Ready
Routing Ready
API Layer Ready
State Management Ready
Testing Ready

Feature Development Unlocked

Recommended Next Step:

/build-feature feature-name

Foundation Lock

The framework must prevent feature commands before foundation completion.

If a user executes:

/build-feature patient-dashboard

before setup is complete:

Display:

Project Foundation Not Found

You must complete:

/setup-project-foundation

before implementing features.

/create-feature-plan

Purpose:

Create a new feature specification file.

Input:

Feature description.

Example:

Create patient notes module.

Behavior:

Generate:

feature-plans/patient-notes.md

Include:

- Business Goal
- User Stories
- UI Requirements
- API Requirements
- Acceptance Criteria
- Technical Design Notes

After creation:

Ask:

Would you like to start implementation now?

Options:

- UI Only
- API Only
- Full Feature
- Later

/build-feature

Purpose:

Implement complete feature.

Input:

Feature file.

Example:

/build-feature patient-dashboard

Behavior:

Read feature-plans/patient-dashboard.md

Implement:

- UI
- API

- Routing
- Validation
- State Management
- Error Handling
- Types
- Tests

Update feature documentation as implementation progresses.

/build-feature-ui

Purpose:

Create frontend implementation only.

Behavior:

Generate:

- Screens
- Components
- Routing
- Mock Data
- Temporary Services

Rules:

No real API calls.

Use temporary service layer.

When API implementation starts later:

Automatically remove temporary implementation.

Preserve architecture.

/build-feature-api

Purpose:

Implement backend integration layer only.

Generate:

- Services
- API Clients
- Hooks
- Types
- Request Models
- Response Models
- Error Handling

Do not create UI.

/update-feature

Purpose:

Update existing feature.

Behavior:

Modify:

- UI
- API
- Tests
- Documentation

Keep implementation synchronized with feature plan.

/update-feature-ui

Purpose:

Update UI only.

/update-feature-api

Purpose:

Update API integration only.

/connect-external-service

Purpose:

Integrate external systems.

Examples:

/connect-external-service keycloak

/connect-external-service stripe

/connect-external-service firebase

Behavior:

1. Ask integration questions.
2. Install packages.
3. Configure environment.
4. Generate services.
5. Update architecture documentation.
6. Generate implementation guide.

/project-status

Purpose:

Display current project progress.

Example Output:

Project Health

Requirements Analysis ✓
Project Foundation ✓

Features

Patient Dashboard ⏳
Appointments ✓
Billing ⏳

Integrations

Authentication ✓
Analytics ⏳
Notifications ⏳

Recommended Next Step:

/build-feature billing

/review-project-architecture

Purpose:

Analyze overall project quality.

Check:

- Folder Structure
- Dependency Rules
- Code Duplication
- Naming Conventions
- Architecture Violations
- Dead Code
- Unused Files

Provide report and recommendations.

/review-feature

Purpose:

Review a feature implementation.

Check:

- Requirements Compliance
- Acceptance Criteria
- Performance
- Accessibility
- Security
- Error Handling

Generate scorecard.

/generate-feature-tests

Purpose:

Generate test coverage.

Create:

- Unit Tests
- Integration Tests
- Component Tests
- API Tests

Based on project testing strategy.

/fix-project-issues

Purpose:

Automatically fix common issues.

Fix:

- ESLint Issues
- Formatting
- Import Issues
- TypeScript Errors
- Architecture Violations

Generate report.

Documentation Requirements

All framework documentation must be beginner-friendly.

Every document must include:

- Purpose
- Why it exists
- Step-by-step instructions
- Examples
- Best Practices

- Common Mistakes
- Troubleshooting

Assume the user is new to the framework.

Documentation must be written in simple, practical language.

Installation Experience

After installation, show a beautiful terminal dashboard.

Display:

Available Commands
Purpose of each command
Recommended next step
Project status

The dashboard should be understandable within 30 seconds by a new developer.

Avoid technical jargon where possible.

The framework should feel like a guided engineering assistant rather than a code generator.

Success Criteria

A developer should be able to:

1. Install the framework.
2. Analyze requirements.
3. Generate architecture.
4. Setup foundation.
5. Build features.
6. Integrate services.
7. Generate tests.
8. Review quality.

without needing to read extensive documentation.

The framework must prioritize maintainability, scalability, consistency, and enterprise-grade React architecture over speed of code generation.